

Understanding GitHub

From Mental Models to Open Source Contribution

A Self-Sufficient Reference

What this deck covers

Part I — Foundations

- Why version control exists
- Mental model: Git vs GitHub
- Core objects: commits, branches, refs
- Local workflow & undoing mistakes

Part II — Collaboration

- Remotes, push/pull, origin
- Branching strategies
- Merge vs rebase, conflicts
- Pull requests & reviews

Part III — Open Source

- Forks & upstreams
- Issues, labels, CI/CD
- Maintainer etiquette

Part IV — Advanced

- History rewriting
- Recovery: reflog, bisect, stash
- Habits, resources, closing thoughts

Why version control exists

The problem: files change over time, and you need to know *what changed*, *when*, *why*, and *by whom* — and be able to go back.

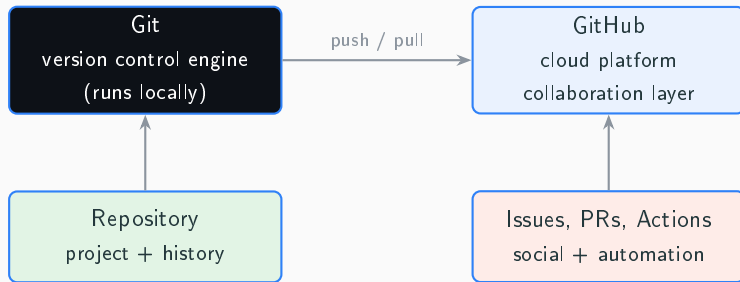
Naive solutions fail at scale:

- `final_v2_FINAL_real.py` — no real history
- Dropbox / Drive — overwrites, no branching, no review
- Email attachments — no merge, no provenance

Git's answer: a **content-addressed snapshot store** with a graph of changes you can branch, merge, inspect, and rewind.

For research: every experiment becomes a checkpoint. For engineering: every change becomes reviewable.

Mental model: Git vs GitHub



- **Git** works without an internet connection.
- **GitHub** is one of several hosts (others: GitLab, Codeberg, Bitbucket).
- You can use Git without GitHub. You cannot use GitHub without Git.

Core objects you should picture in your head

Commit

A snapshot of all tracked files + a message + a parent pointer. Identified by a SHA hash (a3f8c1d...).

Branch

A movable pointer to a commit. Cheap to create (`git branch` is just writing a 40-char hash to a file).

HEAD

A pointer to “where you are right now” — usually to a branch, sometimes directly to a commit (*detached HEAD*).

Working tree

The actual files on disk you edit.

Staging area (index)

A drafting area where you pick *which* changes go into the next commit.

Repository

The `.git/` folder — the database of every object and ref.

Three zones: **working tree** → **staging** → **repository**.

Local workflow & undoing mistakes

```
git init                # start a new repo
git status              # see what's changed
git add path/to/file    # stage a specific file
git add .               # stage everything in current dir
git commit -m "Fix M-value transform for zero-variance CpGs"
git log --oneline --graph --decorate
git diff                # see unstaged changes
```

Safety net — undoing things:

```
git restore <file>      # discard working-tree changes (
                        destructive)
git restore --staged <file> # unstage but keep edits
git commit --amend       # fix the last commit (msg or content)
git reset --soft HEAD~1  # undo last commit, KEEP changes
git reset --hard HEAD~1  # undo last commit, THROW AWAY changes (
                        danger)
git revert <sha>         # NEW commit that reverses an old one (
                        safe for shared history)
```

Remotes: connecting your local repo to GitHub

```
# Clone an existing repo (most common starting point)
git clone https://github.com/user/repo.git

# Or: link a local repo to a new GitHub remote
git remote add origin https://github.com/user/repo.git
git push -u origin main          # -u sets upstream tracking

# See your remotes
git remote -v

# Sync
git push                        # send local commits up
git pull                       # fetch + merge from remote
git fetch                      # fetch only, don't merge
```

Terminology:

- origin — conventional name for *your* primary remote
- upstream — conventional name for the *original* repo you forked from
- main (or master) — the default branch name

Branching: parallel timelines for safe experimentation

```
git switch -c feature/wkde-scoring  # create + switch (modern)
git checkout -b feature/wkde-scoring # equivalent (older)
git branch                          # list local
git branch -a                       # local + remote
git switch main                     # change branch
git branch -d feature/wkde-scoring  # delete (after merge)
```

Why branches matter for research:

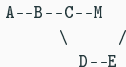
- One branch per experiment / hypothesis / feature
- main stays clean and reproducible
- Failed experiments don't pollute history — just don't merge them
- Cheap and disposable: a branch is a 40-character pointer, not a copy

Merge vs rebase: two ways to combine branches

Merge

Creates a new “merge commit” that ties two histories together.

- **Pros:** preserves true history, non-destructive
- **Cons:** graph can look tangled
- **Use for:** shared branches, public history



Rebase

Re-applies your commits on top of another branch, rewriting history.

- **Pros:** linear, clean history
- **Cons:** rewrites SHAs — dangerous on shared branches
- **Use for:** cleaning up *your own* feature branch before PR

A--B--C--D'--E'

Golden rule: never rebase commits that others have already pulled.

Merge conflicts: not a bug, a conversation

A conflict happens when two branches change the *same lines* of the same file. Git pauses and asks you to decide.

```
<<<<<< HEAD
result = compute_mvalue(beta, epsilon=1e-6)
=====
result = compute_mvalue(beta, eps=1e-5)
>>>>>> feature/wkde-scoring
```

Resolve by:

1. Editing the file — pick one side, combine, or rewrite
2. Remove the ««, ==, »» markers
3. `git add <file>` to mark it resolved
4. `git commit` (for merge) or `git rebase -continue` (for rebase)

Practice tip: deliberately create a conflict in a throwaway repo. Twice. You'll never fear them again.

Pull requests: the scientific peer review of code

A **pull request** (PR) is a proposal: “please merge my branch into yours, here’s what I changed and why.”

Anatomy of a good PR:

- **Title** — imperative mood: *“Add HSIC test for continuous variables”*
- **Description** — what, why, how to test, related issue
- **Small scope** — one logical change; reviewers can’t review 2000-line PRs well
- **Tests** — if the project has them, add or update them
- **Linked issue** — Closes #42

The review loop: reviewer comments → you push new commits to the same branch → PR updates automatically → approve → merge.

Think of a PR as a manuscript submission. The reviewers want it to land.

The fork + upstream workflow

You usually can't push directly to someone else's repo. Instead:

1. **Fork** their repo on GitHub (creates your copy)
2. **Clone** your fork locally
3. Add the original as upstream
4. Create a feature branch, commit, push to your fork
5. Open a PR from your fork → their repo

```
git clone https://github.com/you/their-repo.git
cd their-repo
git remote add upstream https://github.com/them/their-repo.git

# Keep your fork in sync with upstream
git fetch upstream
git switch main
git rebase upstream/main
git push origin main
```

The social & automation layer: issues, labels, CI/CD

Issues

Bug reports, feature requests, questions.

Always search before opening one.

Labels to look for

good first issue, help wanted, bug, documentation. Your entry points.

Discussions & templates

Open-ended Q&A; fill out issue/PR templates properly — it's the difference between attention and an immediate close.

CI = Continuous Integration

Every push runs automated checks: tests, linters, type-checkers, builds. Configured in `.github/workflows/*.yaml`.

CD = Continuous Deployment

Successful builds can auto-publish releases, deploy docs, push packages.

Why this matters for your PR

Status checks must be green. `pytest`, `ruff`, `mypy`, `black` commonly enforced. Maintainers will not merge red builds.

Etiquette: how to actually get merged

Before opening a PR:

- Read CONTRIBUTING.md — it tells you the rules of the house
- Check CODE_OF_CONDUCT.md
- Search existing issues and PRs — yours might be a duplicate
- Open an issue first for non-trivial changes; get buy-in before coding

In your PR:

- Be patient. Maintainers volunteer their time.
- Respond to review comments; don't be defensive
- Squash noisy commits if asked
- Update your branch when conflicts arise — don't expect them to do it

After your PR:

- Stick around. The first contribution is the hardest; the second is easy. The tenth makes you a regular.

Interactive rebase: curating your own history

```
git rebase -i HEAD~5      # edit the last 5 commits
```

Opens an editor with a list:

```
pick    a1b2c3d  Add WKDE  skeleton
pick    d4e5f6g  typo
pick    g7h8i9j  Add tests
pick    j1k2l3m  wip
pick    m4n5o6p  Finalize WKDE scoring
```

Change pick to:

- reword — edit the commit message
- squash (s) — merge into previous commit
- fixup (f) — like squash but discard message
- drop (d) — remove the commit entirely
- reorder lines to reorder commits

Only do this on commits you haven't pushed, or on a personal feature branch.

The advanced toolbox: stash, cherry-pick, reflog, bisect, tag

```
# Temporarily shelve uncommitted changes (e.g. to switch branch)
git stash; git stash list; git stash pop

# Apply a single commit from another branch onto yours
git cherry-pick <sha>

# Reflog: a log of every HEAD movement -- your real safety net
git reflog
git checkout <sha>; git switch -c recovered-branch

# Bisect: binary-search history to find which commit broke something
git bisect start
git bisect bad                # current commit is broken
git bisect good v1.0.0        # this old tag worked
# git checks out the midpoint; you test, then mark good/bad. Repeat.
git bisect reset

# Tag an immutable release point
git tag -a v1.0.0 -m "First release"
git push origin v1.0.0
```


High-level concepts worth internalizing

Concept	Why it matters
Version control	Reproducibility of experiments and results
Branching	Safe parallel experimentation
Commits	Research checkpoints with provenance
Pull requests	Code review as scientific peer review
CI/CD	Automated validation of every change
Forks	Independent lines of work without permission
Tags / releases	Citable, immutable snapshots
Git history	A readable record of how knowledge evolved

Git is, deep down, a tool for thinking carefully about how change happens over time.

Habits that separate beginners from research engineers

Do

- Commit early, commit often
- Write messages your future self will thank you for
- One branch per logical change
- Pull / rebase before starting work
- Use `.gitignore` from day one
- Tag releases of papers / experiments
- Read the diff before you commit

Don't

- Commit secrets (keys, tokens) — they live in history forever
- Commit huge binary files — use Git LFS or DVC
- Force-push to shared branches
- Use `git add .` blindly
- Treat `main` as your scratchpad
- Write messages like "fix" or "update"
- Ignore CI failures

30-day plan & where to read next

Week	Focus
1	Basic Git: <code>init</code> , <code>add</code> , <code>commit</code> , <code>log</code> , <code>diff</code> , <code>status</code> . Version your notes daily.
2	GitHub: <code>push/pull</code> , <code>README</code> , <code>.gitignore</code> . Build a portfolio repo.
3	Branching, merges, deliberate conflicts, your first PR.
4	Open source: fork, sync upstream, contribute one small fix.

References: *Pro Git* book (git-scm.com/book); official docs (git-scm.com/doc); GitHub Skills (skills.github.com).

Entry points: *First Contributions* (firstcontributions.github.io); *Awesome for Beginners*; any repo's good first issue label.

Repos worth reading: [pgmpy/pgmpy](https://pgmpy.org) (relevant to your causal work),

One closing insight

Most people use only:

```
git add    git commit  git push  git pull
```

Advanced users think in:

- history rewriting and curation
- collaboration systems and review culture
- reproducibility of experiments
- workflow design and automation
- safety nets for experimentation

Git is not a tool you finish learning.
It's a medium you keep thinking in.